Building a Simulink model is the perfect way to visually prove the feasibility of your DSP (Digital Signal Processing) and classification logic before compiling it into C++ for the ESP32. By mapping the data flow from the MPU6050 to the LoRa module, you can validate that your thresholds and filters actually work under simulated real-world conditions. 🛠️ +2

Here is your step-by-step guide to building the Pathfinder Seismic AI model in Simulink. 🐘

Phase 1: Setup and Signal Generation (Simulating the MPU6050)

Since you want to prove the logic without walking an actual elephant past your breadboard, we will mathematically simulate the MPU6050's Z-axis output.

1. **Open Simulink:** Type `simulink` in the MATLAB command window and create a **Blank Model**.
2. **Add the "Elephant" Signal:** * Go to the Library Browser > Simulink > Sources.
   - Drag and drop a **Sine Wave** block.
   - Double-click it. Set the *Amplitude* to `3.0` and the *Frequency* to `20 * 2 * pi` rad/s (this represents the 20 Hz low-frequency vibration of heavy footsteps). Set the *Sample time* to `1/200` (200 Hz sampling rate).
3. **Add Background Noise:**
   - Drag another **Sine Wave** block (Frequency: `80 * 2 * pi` rad/s, Amplitude: `1.5`) to simulate passing vehicles.
   - Drag a **Band-Limited White Noise** block to simulate random ground tremors.
4. **Combine Signals:**
   - Drag an **Add** block (Math Operations).
   - Connect all three source blocks into the Add block. The output of this block is your "Raw MPU6050 Z-Axis Data".

Phase 2: Signal Conditioning (The Filter)

Raw data is too noisy for accurate Edge AI classification. We need to filter out the high-frequency vehicle and wind noise.

1. **Add a Digital Filter:**
   - Go to DSP System Toolbox > Filtering > Filter Implementations.
   - Drag the **Discrete FIR Filter** or use a **Lowpass Filter** block.
2. **Configure the Filter:**
   - Double-click the block. Set the *Passband edge frequency* to $50\ \mathrm{Hz}$.
   - This ensures only the low-frequency vibrations (like our 20 Hz target) pass through, while the 80 Hz noise is attenuated.

Phase 3: Feature Extraction (Analyzing the Frequency)

Instead of a complex FFT block which outputs arrays, a highly efficient way to trigger logic in Simulink (and later in TinyML) is to measure the power of the signal in our target frequency band.

1. **Calculate Signal Power:**
   - Drag a **Moving RMS** or **Bandpower** block.
   - Connect the output of your Lowpass Filter to this block.
   - This block calculates the continuous energy level of the filtered 10-50Hz vibrations.

Phase 4: Classification Logic (Edge AI Decision Gate)

Now we build the logic that decides if the vibration is strong enough to warrant an emergency LoRa transmission.

1. **Set the Threshold:**
   - Drag a **Compare to Constant** block (Logic and Bit Operations).
   - Connect the output of the RMS/Bandpower block to it.
   - Double-click and set the *Constant value* to your trigger threshold (e.g., `1.5`). Select the operator `>`.
2. **Create the Alert Trigger:**
   - The output of this block will be a boolean `1` (Danger) or `0` (Safe).

Phase 5: Action and Visualization (Simulating the Output)

We need to visualize the data flow to prove the design works to the judges.

1. **Visualize the Waveforms:**
   - Drag a **Scope** block (Sinks). Set it to have 2 input ports.
   - Connect Port 1 to the Raw MPU6050 Signal (from Phase 1).
   - Connect Port 2 to the Filtered Signal (from Phase 2).
   - *When you run the simulation, you will clearly see the noisy signal become a smooth 20 Hz wave.*
2. **Visualize the Frequency Spectrum:**
   - Drag a **Spectrum Analyzer** block. Connect it to the Filtered Signal to prove that frequencies above 50 Hz have been eliminated.
3. **Simulate the LoRa Transmission:**
   - Drag a **Display** block and connect it to the output of your "Compare to Constant" block.
   - When the simulated elephant steps, the Display will change from `0` to `1`, proving your classification logic successfully triggers the SOS backbone.

Next Step Action: Once your Simulink model is running perfectly, you can use the **Simulink Coder** to automatically generate the C++ code for this exact filter and logic subsystem, which you can directly flash onto your ESP32! Would you like me to guide you through generating that C++ code, or should we move on to the LabVIEW dashboard design?